Spring 2026	C435 Operating Systems ULTIMA 2.0 Phase II – Message Passing (IPC) or U_Pipe (IPC)	H. Hakimzadeh Consult Canvas for Due Date

ULTIMA is a simulated cooperative multitasking operating system. During the last phase you implemented the Scheduler as well as Semaphore. The scheduler allowed you to take control of the state of each task. The semaphore was used to implement mutual exclusion when necessary (e.g., when critical section code needs to be executed). During this phase, your team will design and implement Inter-Process communication using ***message passing*** *facilities to ULTIMA 2.0. Message passing, allows tasks to communicate with each other through the use of send() and receive() calls.*

It is up to you to decide how you would like to implement the above functions. Below are some specification for this phase.

PHASE 3: **Message Passing** (Inter-Process Communication, IPC)

You are to design and implement an inter-process communication system based on message passing. Your server must provide two public functions which are used for message delivery. Message_Send() and Message_Receive(). *A number of other functions will also be needed for internal debugging, status, and clean-up operations.*

```
class ipc {
        struct Message_Type {
                int Message_Type_Id,
                char Message_Type_Description[64];
                etc.
        }

        struct Message {
                int Source_Task_Id;
                int Destination_Task_Id;
                time_t Message_Arrival_Time;          // research time.h, time_t, and tm
                Message_Type Msg_Type;
                int   Msg_Size;                       // lets make these up to 32 bytes long
                char *Msg_Text;
                etc.
        }
        int ipc(int max_tasks);                       // Constructor, return -1 if error
                                                      // create one mailbox/queue per active task.
        int Message_Send(., Message *Message);        // returns -1 if error occurred. Return 1 if
                                                              successful.
        int Message_Send(int S_Id, intD_Id, char *Mess, int Mess_Type);     // overloaded method


        int Message_Receive(int Task_Id, Message *Message);  // returns 0 if no more messages are
                                                              available, loads the Message structure with
                                                              the first message from the mailbox and
                                                              remove the message from the mailbox. Return
                                                              -1 if an error occurs.
        int Message_Receive(int Task_Id, char *Mess, int *Mess_Type);      // overloaded method


        int Message_Count(int Task_id);               // return the number of messages in Task-id's
                                                              message queue.
        int Message_Count();                          // return the total number of messages in all the
                                                              message queues. (overloaded method)

        void Message_Print(int Task_id);              // print all messages for a given Task-id. (but do
                                                              not remove from the queue)
        int Message_DeleteAll(int Task_id);           // delete all the messages for Task_id, from the
                                                              queue.

        void ipc_Message_Dump();                      // print all the messages in the message queue, but
                                                              do not delete them from the queue. (print
                                                              the messages for each task's message queue)
}
```

```
0= text   (Text Message, No action required)
1= service (Request for a service by the task,
           send a notification or result of the
           service back.)
2= Notification (Sent to a task to indicate a
          service was completed.  No action is
           required.)
Etc.
```

Alternatively, you can create a u_pipe (Ultima Pipe) as your Inter-Process Communication mechanism. Your u_pipe must provide a write() method for placing information on the pipe. The pipe can then be read() by another task. Any task should be able to request for the creation of a pipe.

- Once a pipe is created, any task can request to open it for write(). The first task that opens it for write (Source_Task_ID) gets exclusively write access to that pipe.
- Once a pipe is created, any task can request to open it for read(). The first task that opens it for read (Destination_Task_ID) gets exclusively read access to that pipe.

```
class u_pipe {
        int pfd;                                    // pipe file descriptor


        queue<char> pipe_buff(32);                  // queue of characters with up to 32 spaces

                alternatively use an array to represent the buffer, but this may be a little
                more involved, as you would have to keep track of your read and write pointers.
                You also have to make sure the buffer is not overrun, or under-run etc.
                char pipe_buff[32];

                int read_pointer;                   // location of the read pointer
                int write_pointer;


        int Source_Task_Id;                 // initialized when the first task opens the pipe for write
        int Destination_Task_Id;            // initialized when the first task opens the pipe for read

        int u_pipe();                       // Constructor, set up the pipe and returns a pfd
                                            // pfd is a unique sequential number generated each
                                            // time a u_pipe is created.  Returns -1 if error
                                            // set the default value for source_task_id and
                                            // destination_task_id to -1 to indicate no one has
                                            // opened the pipe yet.

        open_for_read(int pfd, int Destination_Task_Id);    // Destination task read from the pipe
        open_for_write(int pfd, int Source_Task_Id);        // Source task writes to the pipe


        close_for_write(int pfd);                           // Source_Task_Id is implicit.
        close_for_read(int pfd);                            // Destination_Task_Id is implicit.


        int write(int pfd, char message);   // write a char to pipe_buff, return -1, if buff is full.

        int read(int pfd, char *message);   // read a char from pipe_buff, return -1, if buff is empty.

                NOTE:   Alternatively, you can block the task, instead or returning -1.
                        Only return -1 if the buffer has been closed by the other side
                        and the user attempts to read from the pipe.

                        Also, return -1 if the user attempts to write to a closed pipe


        void u_pipe_Dump(int pfd);    // print all information in the pipe for this task
}
```

Other Implementation Issues related to IPC class:

- If you are implementing the ipc class, each task must have a mailbox (message queue) associate with it. (this could be a single mailbox per task, or a community mailbox for all tasks.)
    - A single mailbox may be a queue of messages accessible through the task's TCB. **(recommended approach)**
    - A community mailbox may be a community queue for all the tasts. (sending messages may be a little easier, but retrieving message would be more work.
- Access to the mailboxes should be protected (by protection, I mean semaphore) (not necessarily authorization, authentication. However, if you like you can implement that as well. In other words, requests to receive () can be authenticated to ensure the correct task is asking to read the message)

- When a task wants to **send()** a message to another:
    1. The send method must first do a down() on the destination's mailbox semaphore
    2. Send the message (put the message in the queue)
    3. Then do an up() on the destination's semaphore

```
NOTE: the down() and up should be done
in the send() and receive() functions
not by the task.
```

- When a task wants to **receive ()** a message:
    1. The receiver must do a down() on its own (source) mailbox semaphore
    2. Read a message from its message queue /mailbox (and remove it from the queue)
    3. Then do an UP() on the source semaphore.

<u>The Message_Send() function:</u>

1) Allocate a new Message node
2) Copy the message, and other information onto it
3) Place the message in the destination task's mailbox (its message queue).

<u>The Message_Receive() function:</u>

1) Check the count on the message queue. (is this critical section code?)
2) do a down() on the task's mailbox (message_queue) semaphore.
3) De-queue and return the first message from the mailbox (message_queue).
4) do an up() on the task's mailbox.

Other Implementation Issues related to U_PIPE class:

- If you are implementing the u_pipe class, remember that when a task is trying to read or write to the pipe there are a number of items that need to be protected by semaphores (critical section code). These may include the pipe_buffer (array or queue representation), open_for_read() function, open_for_write() function, read() and write() functions, etc.

Suggestions and Hints:

- Think about how the placement of down() and up() could maximize concurrency of your system.

- Consider the concept of MCB (Master Control Block). This may be a convenient method of having access to the scheduler's objects and methods. In addition, you may try to keep your IPC in the MCB. Alternatively, the scheduler object must be registered with the IPC object so that IPC can place messages in the TCB's mailbox.

```
struct MCB {                        //Master Control Block  (it can also be a class)
    scheduler Swapper;
    ipc       Messenger;            //object that provides access to thread ipc objects
    u_pipe    Piper;                //object that maintains a linked list of u_pipe objects
    semaphore  Monitor("Monitor", 1);
    semaphore  Printer("Printer", 1);
}
```

- You may have to add additional functionality to your scheduler to accommodate IPC's access to the TCB's

- Similar to the previous phase, this phase requires some brainstorming by your team. So, do not wait. Start today. Also, please feel free to contact me for discussion and brainstorming sessions.

Note:

- Develop an estimate for how much time this phase requires. Think about all the various components, classes, proof of concepts, external research, group meetings, etc., and associate an amount of time to each. Perhaps a minimum and maximum amount. (that should give a fair average estimate). Then depending on your own assessment of your team's experience, multiply your estimate by a factor (a factor of 2 or 3 is not uncommon). Now you have a little better estimate of the time it will take to do the phase. As you move forward, your ability to estimate will be refined.

Expected Output:

To show that your **ipc** system is working properly, you must show a number of tasks which send, and receive messages. The mailbox for a specific task, (or all tasks) should be shown in each step of the way to ensure the IPC is functioning properly.

Spend some time to think about the best approach for testing this phase. Note that you want your test to be cumulative. In other words, you still want to occasionally dump your scheduler to show that which tasks are running, ready, blocked, etc., who has access to what semaphores, who is in which semaphore queues, etc.

Possible output of the Messenger (MailBox)

Task Number: 1
Message Count: 3
Mail Box:

Source Task-id	Destination Task-id	Message Content	Message Size	Message Type	Message Arrival Time
2	1	Message from task 2 to task 1.	30	Text	xx
3	1	lpr file1	9	Service	xx
1	3	Got your print service request	30	Notification	xx

Alternatively, if you implement **u_pipe**, you must show one or more task creating pipes, opening them, write to them, read from them, etc.

Possible output of the Piper (Streams)

Number of pipes created: 3

pfd	Writer/ Source Task-ID	Reader Destination Task-ID	Write Status	Read Status	Pipe_Buffer Content
0	2	1	Open	Closed	Hello, this is a message from task 2 to task 1.
1	1	2	Open	Open	Message from 1 to 2
2	2	3	Closed	Closed	Stream of data going from 2 to 3

Expected Output:

To show that your **IPC** is working properly, you must show a number of tasks executing, some or all of which sending messages to each other. Along with screen shots showing what is happening to those tasks and their messages in the message queue for each task. (It might be nice to create different scenarios showing a couple of different tasks sending messages to a third task, then have the third task read its messages and display them on its own window. Also using the dump() of the IPC to be able to see what is stored internally)

Spend some time to think about the best approach for testing this phase. Create a short, narrated video of your program execution. Also be prepared to demo your phase in zoom if asked by me.

Managing Your Workflow:

- Each group should create one main folder called ULTIMA, with the following sub folders. Please place this folder in a cloud space that is shared by your team members (and possibly me if I ask for it) so that everyone in your team has access to all the resources.

  ULTIMA

  Introduction
  - Cover page with your group name and number, course # and name, project name, date, group names, e-mails, etc.
  - Project Abstract (Summary of accumulated phase abstracts. With each phase this will be supplemented)
  - Design diagram (Summary of accumulated phase design diagrams. With each phase this will be supplemented)
  - Resume from each team member (use the one from your profile submitted in discussion)

  Phase 1 (Scheduler)

  Phase 2 (Message Passing)
  - Copy of the assignment sheet
  - Phase Abstract
  - External Documentation (for this phase):
    - Design diagram for this phase (including the data structures and class hierarchy, nicely drawn using a drawing package)
  - Source files:
    - With appropriate documentation, each class/module should have a primary author, as well co-authors, if appropriate.
  - Program execution/testing:
    - A Word document showing the submission information including screenshots, code snippets, test cases, etc.
  - Each phase should be completely self-contained (all the files needed to compile and run the phase) (note that after phase 1, all phases are cumulative, in other words, phase 2 depends on phase 1, and it must include phase I.
  - Video with narration for the phase.

  Phase 3 (Memory Management)

  Phase 4 (File Systems)

What to hand in Canvas:

- A zipped copy of the appropriate folders for the phase:
  - Introduction folder
  - Phase 1 folder (Scheduler/Semaphore)
  - Phase 2 folder (IPC)

- A digital version of the completed Self and Peer evaluation form. (DO NOT place this form in the public area, it is not meant to be shared with your team members. This is your communication to me)

Self / Peer Evaluation Form
C435 - Operating Systems
ULTIMA 2.0

Your Name		Team Number:		Team Name:	
Date:		Phase Number:		Phase Title:	IPC

What were your **exact responsibilities** for this phase?
A) Design Activity: (Name the functions, data structures, classes, algorithms you designed or help design)
B) Implementation Activity: (What functions, data structures, classes, algorithms you implemented or help implement)
C) Documentation Activity: (What functions, data structures, classes, or overall project did you document? Documentation could be classified as **internal** to a function, or to a class, or **external** about the phase or the overall project.)
D) Testing Activity: (What functions, data structures, classes, or overall project did you test?) Testing could be classified as unit testing (e.g., function or class) or overall/integration testing (phase level, or project level)
E) Other Activities: (not mentioned above, that you would like to highlight)

How would you improve the system you have implemented? (Be specific)

Do you understand the design decisions that were made by your group? (explain if you agree or disagree with the design decisions)

What did you learn during this phase of the project?

Which of the following helped you to understand and complete this phase of the project? (e.g., Textbook, Class videos/PowerPoint, labs, class discussion, group discussions, external sources). If external sources, please specify.

Given the following categories, evaluate yourself and your team member. Estimate the contributions based both in terms of hours and percentages of total work. *(For example: I attended all the meeting 100%, 4 hours, I implemented a small class, but the rest of the classes were implemented by my team members, so I estimate my contribution as 25% and 3 hours, I did most of the documentation, so my contribution was 90% and spent 3 hour)*

YOU	Estimated # of Hours Spent (during the phase)	Estimated % of your contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		

Your Team Member's Name:	Estimated # of Hours Spent (during the phase)	Estimated % of her/his contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		

Your Team Member's Name:	Estimated # of Hours Spent (during the phase)	Estimated % of her/his contribution to the overall work
a) Attending group meetings		
b) Participation in the design		
c) Participation in the implementation		
d) Participation in the debugging		
e) Participation in testing		
f) Participation in the documentation		
g) Overall contribution to the group		
h) Other Activities (Explain below):		